一、1. 进程虚拟机（以JVM为例）

(1) 进程虚拟机并非虚拟一个完整计算机，而是在操作系统进程中模拟一个虚拟执行环境，用来运行中间代码，并且此环境受管理

(2) 特点：

① 跨平台性：一次编译，到处运行。可屏蔽底层操作系统和硬件差异

② 抽象层次高：虚拟指令集（字节码）和运行环境，不虚拟硬件

③ 性能可接近原生性能

④ 安全性、隔离性：多个JVM实例之间隔离性差，但是与宿主系统隔离良好

2. 操作系统虚拟机 ……（∴KVM）

(1) 原生虚拟机：直接运行在物理硬件之上，"第一层软件"

特点：① 性能高：直接控制硬件

② 轻量级，专注于虚拟化任务，自身开销小

③ 隔离性强，安全性高

(2) 主机虚拟机：安装在宿主操作系统之上，"第二层"软件

特点：① 性能相对较低：请求硬件需经宿主操作系统中转处理

② 资源与宿主机其他应用程序共享

③ 隔离性强，安全性高

二、1. 目录服务器

(1) 有一专门的目录服务器来维护当前可用服务列表

先查询目录服务器，得服务地址，再连接实际节点。

(2) 优 ① 管理集中，方便安全访问控制。

② 解耦：客户端与服务提供者不耦合，服务地址可变更对客户端透明

缺：① 易故障，目录服务器故障整个系统宕机

② 性能较低

③ 维护复杂

2. 超级服务器

(1) 维护一个进程监听多个端口，根据请求启动相应服务进程

(2) 优：① 结构简单，易于维护和实现

② 集中管控，统一处理

缺：① 单点故障风险高

② 扩展性差，易成为瓶颈。

第四讲作业 23336179 子福象

一、(1)关系：①两者是不同抽象层次的通信范式，消息通信更基础，更通用。
RPC是在消息通信基础上的一种高级通信模型。
②消息通信和RPC在现代分布式系统中协同工作。

(2)同：①功能目的相同：都是为了实现分布式系统进程通信
②都基于网络通信协议等。

(3)异：①模型：RPC是请求-响应。消息通信是简单消息传递模型。
②耦合性：RPC是紧耦合。后者则是松耦合支持离线处理。
③时序性：RPC为同步调用为主，后者以异步处理为主。

二、(1) Gossip多播
①原理：节点每次随机挑选k个邻居"传染"消息，不依赖全局拓扑，去中心化。
②适用于：节点规模大，动态进出频繁。
只要求最终一致性，可容忍短期不一致。
消息体较小，冗余重复代价低
③例子：P2P文件共享的元数据同步。

(2)基于Overlay的结构化多播
①原理：先构建逻辑树/环，消息沿树边分发。
维护代价高，但路径短，冗余少
②适用于：节点规模不大，相对静态，拓扑变化频率低；
追求低延迟，低冗余。
消息体较大，
③例子：视频会议；金融行情多播…

第五讲作业 马福泉 23336179

一、新节点插入与指状表更新

1. 新节点首先向网络中任意一个已知节点发起请求从而进入Chord环

2. 通过上述引导节点查找前驱与后继，即新节点执行 find_successor(newnode)找到环上第一个ID大于等于新节点ID以及最后一个ID小于新节点ID的节点

3. 建立前后关系。新节点将自己的successor指向后继节点，predecessor指向前驱节点；与此同时，原后继节点的前驱更新为新节点，原前驱节点的后继更新为新节点，

4. 构建指状表：

新节点根据自身ID，计算其Finger Table的m个表项 (m=5)

finger[i].start = $(n+2^{i-1}) \bmod 32$, $i=1,2,3,4,5$

对每个表项，新节点通过find_successor(finger[i].start)查找并填入"环上第一个ID大于等于 finger[i].start 的节点，作为表项的指向节点

5. 更新其他节点的指状表：这个步骤是可选的、渐进的，若已有节点的指状表中某些项本应指向新节点，则这些节点后续可通过查询或通知机制更新其相应的指状表项，从而优化路径。

二举例：

m=5，ID范围 0-31，新节点 N=14 加入 Chord 环。

1、节点 14 联系节点 13，查询 find_successor(14)，得后继为 15，前驱为 13

2、节点 14 设置 successor=15，predecessor=13。

通知节点 13 与节点 15 更新前后关系

3、节点 14 计算 5 个指状表项起始位置

finger[1].start = 14+1=15，查找 successor(15)

finger[2].start = 14+2=16，查找 successor(16)

finger[3].start = 14+4=18 查找 successor(18)

finger[4].start = 14+8=22 查找 successor(22)

finger[5].start = 14+16=30 查找 successor(30)

4、通过向网络查询，节点 14 填充自己的指状表，完成填入

5、其他节点，若有某些 finger 指向 15，则后续逐步更新。

第六讲作业. 马福泉 23336179

1、a→b，事件b的发生不一定与a有关系。(在现实中)

在分布式系统中，a→b是一种逻辑上的因果顺序，用于通过时间戳，消息传递等机制来判断事件的先后与依赖关系。这是分布系统中为了维护一致性和顺序性而引入的逻辑概念，并不等同于现实世界中的因果关系。

2、(1)场景定义：N_1 N_2两个节点，协同编辑文档D，支持并发编辑

需解决：①操作因果顺序追踪 ②并发冲突检测

(2)操作算法过程：①初始化：每一用户节点维护一个向量时钟，长度=节点数，(初始为0

②操作与传播

N_1编辑：用户输入"Hello"，记为op1

N_1递增自身计数器，$VC_1=[N_1:1, N_2:0]$

并把op1与VC_1绑定，广播为$e_1=(op_1, VC_1)$

同理：N_2编辑：用户删除该行，记为op2

N_2递增VC_2的N_2 $VC_2=[N_1:0, N_2:1]$

op2与VC_2绑定，广播为$e_2=(op_2, VC_2)$

③接收处理与冲突检测：

如：N_2收到e_1对比VC_1与VC_2

两者互不包含，说明op1与op2并发且产生了冲突

提示用户手动选择编辑解决冲突

(3)向量时钟的作用：

①因果追踪保障：可以记录操作OP

②可以检测冲突：当2个VC互不包含，说明并发冲突

③可以保持一致性：合并VC可实现文档协同一致编辑。

3. (1) 第二次互斥算法为：基于 Lamport 逻辑时钟的全序多播。

它与 Ricart & Agrawala 分布式算法最相近。

(2) 相同点：① 维护请求队列，时间顺序处理请求

② 分布式协作：无需中央协调器，进程间通过消息交换协作实现互斥访问权。

(3) 不同点：① 前者通过队列和超时机制可实现高容错性，后者对单进程故障敏感，易阻塞，容错率低。

② 请求机制不同：前者通过全序多播广播请求，需要等待所有进程的 ALLOW 消息，依赖逻辑时钟实现全序。后者直接向其他所有进程发送请求，接收同意信息后进入临界区

③ 复杂度：前者需维护多播队列

后者算法简单，但要处理更多消息。

4. (1) 消息唯一性：每个 Election 消息携带发起点 ID（全局唯一）以及序列号（发起点本地递增，全局唯一），从而标识消息来源与版本。

(2) 相遇时比较优先级：

当节点收同时收到两个 ID 发来的 Election 消息：保留 ID 更大的消息，丢弃低 ID 消息

(3) 状态管理

节点维护一个已处理消息表（记录 ID、序列号）

避免重复处理，

丢弃低 ID 消息：可直接标记为“已处理”，加入到上述已处理消息表。

马福泉 23336179 第九讲作业.

Q1: (1)强一致性：任何读操作都能立即看到最近的写操作结果，
所有节点的操作顺序严格一致
最终一致性：无新写操作后，所有副本最终会一致，但不保证实时
强一致，允许短暂不一致窗口。

(2)优势价值：性能好。例：AWS S3存储桶使用最终一致性，支持高频访问。
扩展性好。例：Redis Cluster，支持动态扩缩容...
容错性好。Cassandra在网络分区时优先响应读写请求。

Q2 P3：x最新写是 P1.W(x)a，返回a
P4：y最新写是 P2.W(y)b. P2写已经通过P3读操作传播，返回b

Q3：每个数据副本维护版本向量跟踪操作因果依赖，为每个操作分配逻辑
时间戳，反映因果偏序。反熵同步修复副本差异，综合因果顺序同步，
读写携带因果元数据验证合法性。

23336179 马福泉 第八讲作业。

1. 三模冗余不能处理拜占庭故障。

① 三模冗余（TMR）是一种通过三个独立模块并且执行相同运算，并采用多数表决机制实现容错的技术

② 拜占庭故障是指组件可能以任何方式失效，如恶意发送矛盾和错误信息。

③ TMR多数表决的容错机制是建立在故障模块输出是真实一致的，但拜占庭故障中，故障模块可能向不同副本发送不一致的数据，TMR表决器就无法基于简单比较得出正确结果。TMR仅适用于非恶意环境下的单点故障容错

2. 在可靠多播中，通信层通常都需要保留消息副本用于重传。

① 接收方若检测消息丢失，会向发送方发重传请求（NAK）。
所以发送方需保留消息副本方便重传。

② 某些高级协议（如分层的多播）可能将重传责任转移给其他接收者，但该协调者仍需在系统中保留消息副本以便于重传

3. ① 不确定上下文状态：新协调者选举后，可能无法获取故障协调者的最终决策上下文，从而无法提交或中止事务。

② 部分提交信息不完整导致不一致。如果协调者在提交阶段仅向部分参与者发送提交指令后故障，已提交的参与者无法回滚，而未收到的参与者会继续阻塞

③ 网络分区或延迟也可能导致消息无法及时送达，影响阻塞解除。

4. 无状态服务器不需要执行检查点。

① 无状态服务器不会在本地维护任何状态，每次请求都是独立的，且包含处理所需的全部信息。

② 无状态服务器也许会在本地缓存了临时中间状态，但这不属于检查点范畴，而且由外部系统管理，而非服务器本身。